

ICare Security Assessment

SE4030 - Secure Software Development | Group assignment | Prepared 25 September 2026

Submission metadata

Team: TODO - four member names, index numbers, and individual contributions. Modified GitHub repository: TODO. YouTube demonstration (maximum 20 minutes): TODO. Semester start date: TODO. These details were not supplied and must be completed before submission.

Original project: <https://github.com/kojithan-y/Optical-Store-Management-System> . Baseline commit: 5b5b25869d92a662b1e5180eb1b4f22bcf3a7830, dated 11 February 2025. Confirm that this predates the semester start. ICare is an optical store application with React, Express, MongoDB, appointments, prescriptions, tickets, inventory, orders, and administrator workflows.

Method and scope

White-box review of the checked-in baseline source and targeted regression tests of the hardened code. Findings below are supported by identifiable original routes and data flow. No external scanner result, live database exploit, penetration test, live Google sign-in, or dependency audit is claimed. Use an isolated copy of the baseline and test data for demonstrations.

Executive summary

Twelve distinct weaknesses were identified across destructive setup endpoints, authorization, payment integrity, privacy, reset tokens, file upload, and error handling. The highest impact issues allowed public deletion of users/products, cross-user order access, arbitrary paid status, and storage of payment card data. Code fixes are implemented. Operational work remains for historical secrets and card records.

Findings and remediation

V01 Public database reset

Original: GET /api/seed in seedRoutes.js deletes all Product and User documents and repopulates default accounts without authentication.

Risk: Unauthenticated database destruction and known demo credentials.

Fix: Removed /api/seed from server.js. Seed data is no longer reachable through HTTP.

Evidence: Baseline: ICare/backend/routes/seedRoutes.js and server.js; modified: server.js.

V02 Order IDOR

Original: GET /api/orders/:id checked only for a valid bearer token and returned any order by its ID.

Risk: A customer could obtain another customer's address, items, and payment state.

Fix: Require matching order.user or a current administrator before returning an order.

Evidence: Baseline and modified: ICare/backend/routes/orderRoutes.js.

V03 Unauthorized payment and delivery changes

Original: PUT /api/orders/:id/deliver and /pay required only sign-in. The card branch marked an order paid without processor confirmation.

Risk: Any user could alter fulfilment or forge a paid order.

Fix: Both routes now require administrator. The pay route records only collected Cash on Delivery payment and rejects repeated/non-COD updates. The customer Pay Now control was removed.

Evidence: Baseline and modified: orderRoutes.js; frontend OrderScreen.js and PaymentListScreen.js.

V04 Client-controlled order totals

Original: POST /api/orders copied client order item prices and totals into MongoDB.

Risk: A customer could submit a lower item price or total.

Fix: Lookup catalog products by ID, validate quantity and stock, copy server-side prices and recalculate shipping, tax and total.

Evidence: Modified: orderPricing.js and security.test.js. Test submits price 0.01 and confirms catalog price 200.

V05 Stored card data and CVV

Original: CardDetailsScreen saved PAN and CVV in localStorage; cardRoutes.js and orderModel.js stored them in MongoDB.

Risk: Sensitive payment credentials could be exposed through XSS, browser access, database breach, or API response.

Fix: Removed card form, saved-card API, card schema/model and order card fields. Checkout now supports Cash on Delivery only.

Evidence: Baseline: CardDetailsScreen.js, cardRoutes.js, cardModel.js, orderModel.js. Modified: PaymentMethodScreen.js and orderRoutes.js.

V06 Prescription access control

Original: Prescription list, detail, create and update routes accepted any authenticated user.

Risk: Medical prescription data could be read or modified by unrelated customers.

Fix: Applied administrator checks to prescription management routes.

Evidence: Baseline and modified: ICare/backend/routes/prescriptionRoutes.js.

V07 Stale role in JWT

Original: isAuth trusted isAdmin from a 30-day token even after a user was demoted or deleted.

Risk: Former administrators retained privileges until their token expired.

Fix: isAuth now loads the current account and role from MongoDB on every request.

Evidence: Baseline and modified: ICare/backend/utlis.js.

Further findings

V08 Sensitive user fields in API responses

Original: Admin user list/detail returned full Mongoose documents, including password hashes and resetToken. Update returned the full saved user.

Risk: Extra credential material could leak through a compromised admin browser or logs.

Fix: Exclude sensitive fields in list/detail queries; use a small update response object.

Evidence: Baseline and modified: ICare/backend/routes/userRoutes.js.

V09 Unsafe password reset

Original: Reset JWT appeared in server logs, lasted three hours, was not cleared after use, and unknown emails returned 404.

Risk: Reset links could be reused or exposed; account enumeration was possible.

Fix: Generate random token, store only SHA-256 hash for 15 minutes, clear after reset, avoid logging it and return generic request response.

Evidence: Baseline and modified: userRoutes.js and userModel.js.

V10 Unbounded file upload

Original: multer() used memory storage without a size or type limit.

Risk: An administrator upload could consume excessive memory or send unwanted file types to Cloudinary.

Fix: Limit to one 5 MB JPEG, PNG or WebP upload and reject missing files.

Evidence: Baseline and modified: ICare/backend/routes/uploadRoutes.js.

V11 Ticket response IDOR

Original: POST /api/tickets/:id/responses allowed any authenticated user to reply to any ticket.

Risk: Private support conversations could be modified by other customers.

Fix: Allow ticket owner or current administrator only.

Evidence: Baseline and modified: ICare/backend/routes/ticketRoutes.js.

V12 Internal error disclosure

Original: Global error handler returned err.message as a 500 response.

Risk: Database and implementation details could be disclosed.

Fix: Return generic 500 messages and structured 400 validation errors; log server-side.

Evidence: Baseline and modified: ICare/backend/server.js.

Google OpenID Connect feature

User journey

Sign In now offers Sign in with Google. A successful first sign-in creates a non-admin ICare account bound to the Google subject (sub); later sign-ins resolve the account by sub. An existing password account with the same email is not auto-linked, preventing email-based account takeover. The app then issues its existing ICare bearer token.

Protocol and trust boundaries

The backend starts an authorization-code flow with openid email profile scopes. It sets short-lived HttpOnly SameSite=Lax state, nonce and PKCE verifier cookies. The callback compares state, exchanges the code with Google over HTTPS using the configured client secret and verifier, retrieves Google public keys, validates the RS256 ID-token signature and issuer, audience, expiry, nonce, subject and verified email, then clears flow cookies. The ICare token is delivered to the React callback via URL fragment and removed from browser history immediately.

Configuration and verification

Create a Google Cloud Web application OAuth client. Register <http://localhost:3000/api/auth/google/callback> for the React dev server, or the equivalent HTTPS production URI. Populate `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET` and `GOOGLE_REDIRECT_URI` in an untracked backend `.env`. Automated tests cover accepted identity and rejected signature, audience, nonce and expiration. A live Google end-to-end test requires real credentials and has not yet been performed.

Verification and remaining work

Completed checks

Backend: npm test - 4 tests passed (catalog price tampering, stock/duplicate rejection, ID-token verification, stale administrator claim). Frontend: npm run build completed with existing ESLint warnings. Node syntax checks passed for changed backend files. The checks do not replace authorization integration tests against MongoDB or a live provider test.

Unresolved and operational risks

1. Original Git history contains backend `.env`. The modified working tree removes it from tracking, but old commits still contain it. Rotate MongoDB, JWT and Cloudinary credentials and consider history rewrite before publication. 2. Historical card records remain in any deployed MongoDB and backups; purge them securely under an approved retention procedure. 3. Card payments remain unavailable until a real PCI-compliant hosted gateway with server-side verification is integrated. 4. Existing bearer tokens live in `localStorage` and may be stolen by future XSS; migrate to a short-lived secure HttpOnly session architecture after a full frontend review. 5. A full dependency update, external scan, rate limiting and integration test suite remain future work.

Process improvements

Add threat modelling for customer/staff data, an authorization matrix for every route, code review for payment and reset flows, secret scanning and `.env` ignore rules, regression tests for role/owner boundaries, dependency scanning, and a deployment checklist for data migrations and credential rotation.

References

OWASP Top 10: <https://owasp.org/www-project-top-ten/>

Google OpenID Connect: <https://developers.google.com/identity/openid-connect/openid-connect>

Google OAuth 2.0 web server flow: <https://developers.google.com/identity/protocols/oauth2/web-server>

Google ID-token verification: <https://developers.google.com/identity/gsi/web/guides/verify-google-id-token>